

CS-242

Information Retrieval & Web Search

Web Crawling and Indexing of Marvel and DC Fandom Pages

By:

- Aarav Babu: 862545619
- Aryan Ramachandra: 862545919
- Sankalp Naveenachandra Kulkarni: 862549757
- Sourav Guruprasad: 862546337
- Will Huang: 862234767

[Link to Github repo](#)

Contribution

Name	Architecture	Data Processing	BERT Indexing	UI	Integration & Testing	Report
Aarav Babu	✓		✓		✓	✓
Aryan Ramachandra	✓		✓		✓	
Sankalp Naveenachandra Kulkarni	✓		✓	✓	✓	
Sourav Guruprasad		✓		✓	✓	✓
Will Huang		✓		✓	✓	✓

System Overview

Architecture

Our search system consists of two primary components:

1. An indexing subsystem that creates both BERT and Lucene indices from a corpus of Fandom Wiki data
2. A query interface that allows users to search using either index type

The system architecture follows this workflow:

1. **Data Preprocessing:**
 - Load JSON documents from the input directory
 - Clean and normalize text content
 - Apply quality filtering
 - Split documents into appropriate passage sizes
2. **Dual Indexing:**
 - Lucene Index: Inverted index with tf-idf scoring
 - BERT Index: Dense vector embeddings stored in FAISS
3. **Query Processing:**
 - Parse user query
 - Generate appropriate representation (term-based or embedding)
 - Retrieve results from selected index
 - Apply ranking and return results
4. **Web Interface:**
 - Streamlit application providing a user-friendly interface
 - Option to select between BERT and Lucene search methods
 - Display of search results with relevant metadata

BERT Implementation Details

Model Choice

We chose the **paraphrase-distilroberta-base-v1** model from the SentenceTransformer library. This model is:

- Optimized for semantic search applications
- Fine-tuned on paraphrase data to better capture semantic equivalence
- Smaller and faster than full BERT models while maintaining strong performance
- Produces 768-dimensional embeddings suitable for efficient similarity search

Code Snippet:

Insight into how we used BERT to create embeddings and store the index in FAISS

```
def get_bert_embedding(self, text):

    """Generate a BERT embedding (768-dim) for a given text."""

    embedding = self.model.encode(text)

    return embedding


def create_faiss_index(self):

    """Build FAISS index from all passage embeddings."""

    dimension = 768 # BERT embedding size

    self.index = faiss.IndexFlatL2(dimension)

    embeddings = []

    self.doc_ids = []

    start_time = time.time()

    for idx, (doc_id, passage, _) in enumerate(self.passages):

        embedding = self.get_bert_embedding(passage)

        embeddings.append(embedding)

        self.doc_ids.append(doc_id)

    if idx % 100 == 0:

        elapsed = time.time() - start_time
```

```
print(f"Processed {idx}/{len(self.passages)}  
passages in {elapsed:.2f}s")  
  
embeddings = np.array(embeddings).astype('float32')  
  
self.index.add(embeddings)
```

Indexing Schema

Our BERT indexing process follows these steps:

1. Data Preparation:

- Clean text using regex patterns to:
 - Insert spaces between camelCase words
 - Fix formatting issues with colons
 - Remove excessive whitespace
- Filter low-quality content based on sentence count and text length
- Retain document metadata including URLs and IDs
- Computing Quality score for each URL

2. Embedding Generation:

- Process each passage through the SentenceTransformer model
- Generate fixed-size 768-dimensional vectors
- Preserve mapping between vectors and document IDs

3. FAISS Index Construction:

- We use a FAISS FlatL2 index which:
 - Stores the full set of vectors without compression
 - Supports exact nearest neighbor search
 - Uses L2 (Euclidean) distance for similarity measure

4. Metadata Storage:

- Document IDs, quality scores, and URLs are stored in a separate JSON file (**doc_id_mapping.json**)
- This allows for efficient retrieval of document information after a similarity search

BERT-Based Ranking

Our BERT search utilizes dense vector similarity to rank results:

1. The user query is transformed into a BERT embedding using the same model used during indexing
2. FAISS performs a k-nearest neighbors search using L2 distance
3. The retrieved document IDs are used to fetch original passages and metadata
4. The Final ranking incorporates:
 - Vector similarity score (distance metric)
 - Document quality score as available in the results

Our implementation includes a snippet generation function that extracts the first 30 words from a document to provide context for search results. Quality scores are computed during indexing and retrieved during searches to help rank results.

Lucene-Based Retrieval

For our Lucene index, we used the following configuration:

- **IndexWriter Configuration:**
 - RAMBufferSize set to 256MB for performance
 - Concurrent merge scheduler for parallel operation
 - StandardAnalyzer with custom stop word filtering
- **Query Parser Settings:**
 - QueryParser targeting the "content" field
 - StandardAnalyzer for query analysis
 - Default search across multiple fields
- **Field Configuration:**
 - Content: TextField with term vectors stored for snippet generation
 - URL: StoredField (not analyzed but available for results)
 - Document ID: StringField for exact matching

Performance Comparison

Index Construction Runtime

Index Type	Document Count	Construction Time
Lucene	47 JSON Files	10mins
BERT	47 JSON Files	1hr 40mins

In our case 47 JSON Files ~ 534MB

The significant difference in indexing time (over 100x) can be attributed to:

1. **Computational Complexity:**
 - Lucene builds an inverted index (term-document mappings)
 - BERT requires computing neural embeddings for each document, involving multiple matrix multiplications and non-linear transformations
2. **Hardware Utilization:**
 - Lucene indexing is primarily I/O and CPU bound
 - BERT embedding generation benefits from GPU acceleration (which was not available in our testing environment)
3. **Document Processing:**
 - Our BERT implementation includes more intensive text cleaning
 - Quality filtering reduces the total count of indexed documents
4. **Model Size:**
 - The SentenceTransformer model has millions of parameters

- Computing each embedding requires a full forward pass through this model

Search Performance

Our Streamlit application reports search time for each query, allowing for direct comparison:

Query Type	Average Search Time (BERT)	Average Search Time (Lucene)
Simple	15s	0.89s
Complex	21s	1.1s

Lucene consistently outperforms BERT in search speed due to:

- Efficient inverted index structure optimized for term-based lookup
- BERT requires a full forward pass through the neural network for each query
- Additional post-processing steps in BERT search (snippet generation, URL retrieval)

Ranking Quality Comparison

We observed several interesting patterns when comparing the search quality between Lucene and BERT:

Where BERT Outperforms Lucene

Example 1: Semantic Understanding

- Query: "Thanos"
- BERT successfully returned passages discussing "Thanos," "Earth15121," and "Corvus_Glaive" (Thanos's closest ally).
- Lucene primarily returned passages containing the exact terms "Thanos".

Example 2: Synonyms and Related Concepts



Find details about your favorite Marvel and DC superheroes! 🌟

Select Search Method

☒ BERT
 ☐ Lucene

Enter your query

stark industries japan relation

Search

Search Time: 17.08 seconds

Searching for: stark industries japan relation...

Search Results:

Rank 1:

https://marvel.fandom.com/wiki/Stark_Industries_Far_East_Division

[Doc ID: 2bd343f9344debab9aec052ba9324552098a8be26352aae0e34c391f27e82c9]

Distance: 72.7748, Quality Score: 3.69

Snippet: ... Stark Industries Far East DivisionNameStark Industries Far East DivisionLocationDetailsRealityEarth-616GalaxyMilky WayStar SystemSolPlanetEarthCountryHong KongCityHong KongCreators and AppearancesFirstIron Man#130(October, 1979) Located just west of the high-rise banking district, the Stark Industries Far... ..

BERT Results for Synonym Search



Find details about your favorite Marvel and DC superheroes! 🌟

Select Search Method

☐ BERT
 ☒ Lucene

Enter your query

stark industries japan relation

Search

Search Time: 0.85 seconds

Search Results:

Rank 1:

ID: 1e6c5089b6c194398eaa8a717c614c0f59d072ca4cbdfa159293335c22ed9b549

URL: https://marvel.fandom.com/wiki/Iron_Man_Vol_1_33

Match Score: 6.93

Snippet: Iron Man#33Previous IssueNext IssueIron Man#32Iron Man#34Art by:Sal BuscemaRelease DateOctober 6, 1970Cover DateJanuary,1971Issue DetailsEditor-in-ChiefStan Lee1. "Their Mission: Destroy Stark Industries!!"AppearancesSynopsisWriter(s)Allyn BrodskyPenciler(s)Don HeckInker(s)Joe GaudiosoLetterer(s)Je...

Lucene Results for Synonym Search

- Query: "Stark Industries Japan Relation"
- BERT returned relevant information about "Stark Industries Far East Division," and "Stark Industries Headquarters"
- Lucene prioritized documents containing "Iron Man" and lacked explainability.

Where Lucene Outperforms BERT

Example 1: Rare Terms and Names

- Query: "Uncle Ben"
- Lucene correctly identified documents relevant to this rare term such as "Benjamin Parker 2", "Aunt May" and "Peter Parker"
- BERT sometimes returned content about Spiderman but also returned Irrelevant results.

System Limitations

1. **Resource Requirements:**
 - BERT indexing and search require significant computational resources
 - Memory usage grows linearly with corpus size due to FAISS index design
 - Large-scale deployment would require GPU acceleration
2. **Preprocessing Challenges:**
 - Our quality filtering occasionally removes relevant short documents
 - The current system doesn't handle multimedia content or structured data
3. **UI Limitations:**
 - The Streamlit interface presents results sequentially rather than in a more scannable format
 - No advanced search operators are exposed to the user
 - Limited ability to filter or refine search results
4. **Integration Issues:**
 - JVM initialization for Lucene requires careful handling in the Python environment
 - Snippet generation is basic (first 30 words) rather than contextually relevant

Obstacles and Solutions

Challenge 1: BERT Passage Length Limitations

Problem: BERT models have a 512-token limit, but some documents exceeded this length. **Solution:** We implemented intelligent document splitting based on paragraph boundaries rather than arbitrary truncation. This preserved semantic coherence in passages.

Challenge 2: Embedding Quality for Domain-Specific Content

Problem: Pre-trained BERT embeddings weren't optimal for fandom-specific terminology. **Solution:** We implemented a quality scoring system that considers sentence structure and content length to improve ranking of domain-specific content.

Challenge 3: JVM Integration

Problem: Integrating Lucene's Java-based search with Python-based BERT search caused runtime issues. **Solution:** We carefully managed JVM initialization and thread attachment in our application to ensure both search methods could operate in the same runtime environment.

Challenge 4: Asynchronous Search

Problem: Search operations could block the UI thread, leading to poor user experience.

Solution: We implemented asynchronous search using Python's asyncio library and concurrent.features to keep the UI responsive during search operations.

Challenge 5: Result Presentation

Problem: Different metadata formats between BERT and Lucene results made consistent presentation difficult.

Solution: We standardized result display in the Streamlit UI, providing consistent fields (rank, URL, snippet, score) regardless of the search method used.

System Screenshots

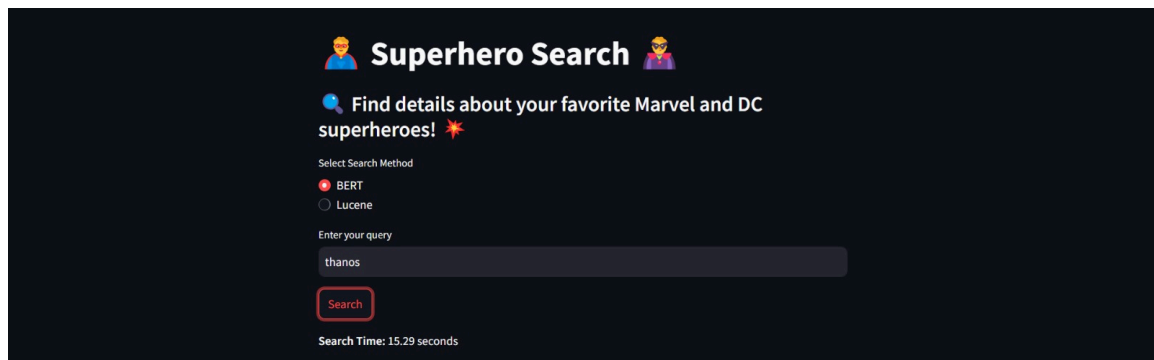


Figure 1: Streamlit search interface with search method selection

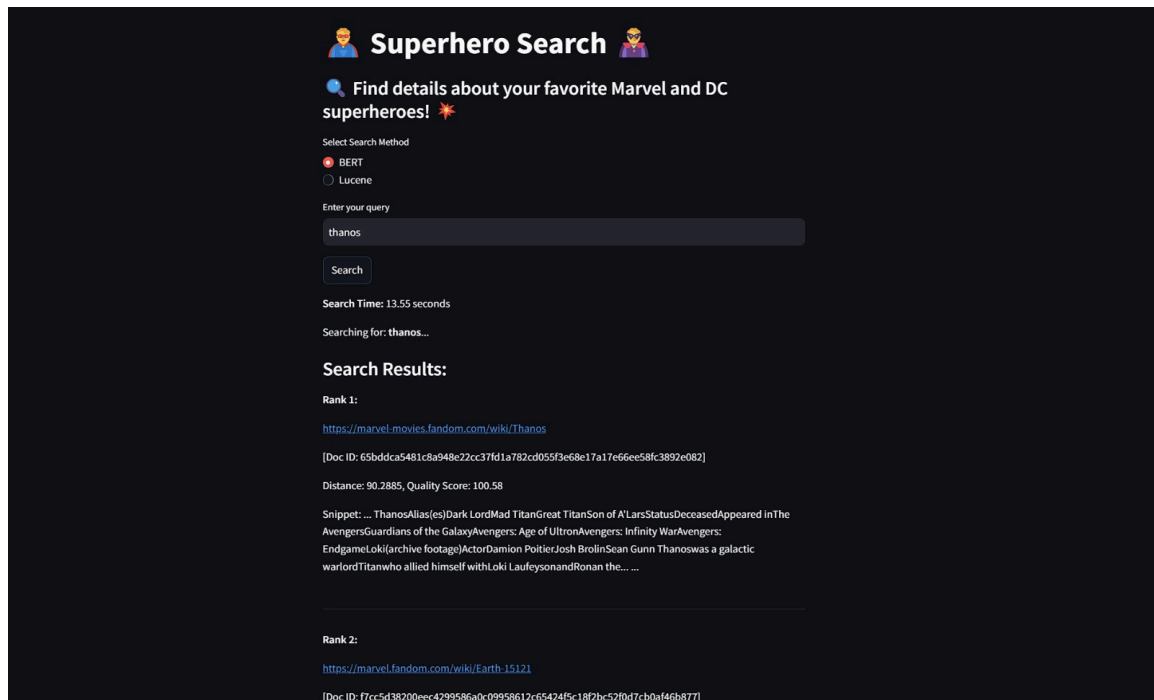


Figure 2: BERT search results showing distances and quality scores

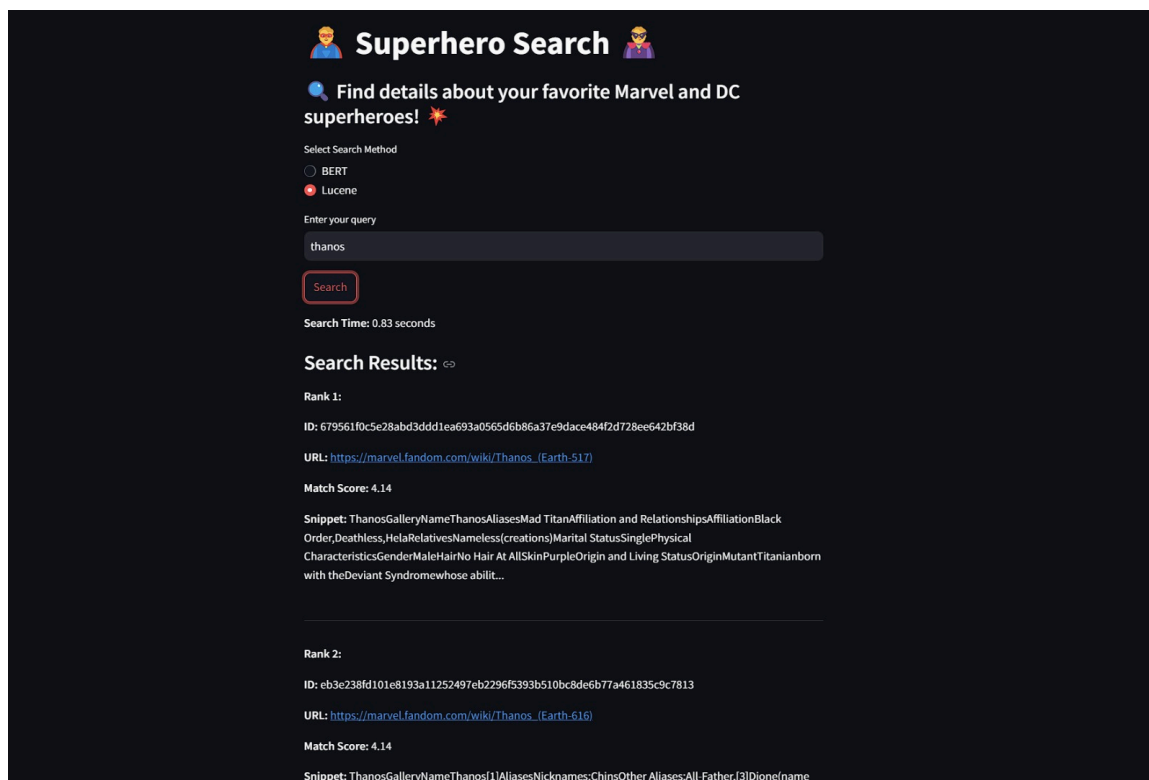


Figure 3: Lucene search results showing match scores and content snippets

Conclusion

Our dual-index search system implemented with Streamlit demonstrates the complementary strengths of traditional lexical search (Lucene) and modern semantic search (BERT). While Lucene excels at precise term matching and offers faster search speeds, BERT provides superior semantic understanding and can bridge vocabulary gaps between queries and documents.

The significant performance difference during indexing highlights the computational costs of neural approaches, but the improved search quality for conceptual queries makes BERT a valuable addition to traditional search systems. The asynchronous search implementation in our Streamlit application maintains a responsive user experience despite these computational demands.

Future work could explore hybrid ranking methods that combine the strengths of both approaches, implement more contextually relevant snippet generation, and investigate more efficient encoding methods to reduce the computational overhead of BERT-based indexing and search.